SMART-CITY ROAD NETWORK

Shortest Path using Dijkstra's Algorithm in C++

Problem Statement

Develop a C++ program to represent a smart-city road network as a weighted graph with non-negative distances. Apply Dijkstra's algorithm from a user-selected junction, display the shortest distances to all reachable junctions in ascending order, and show the shortest path to a selected junction.

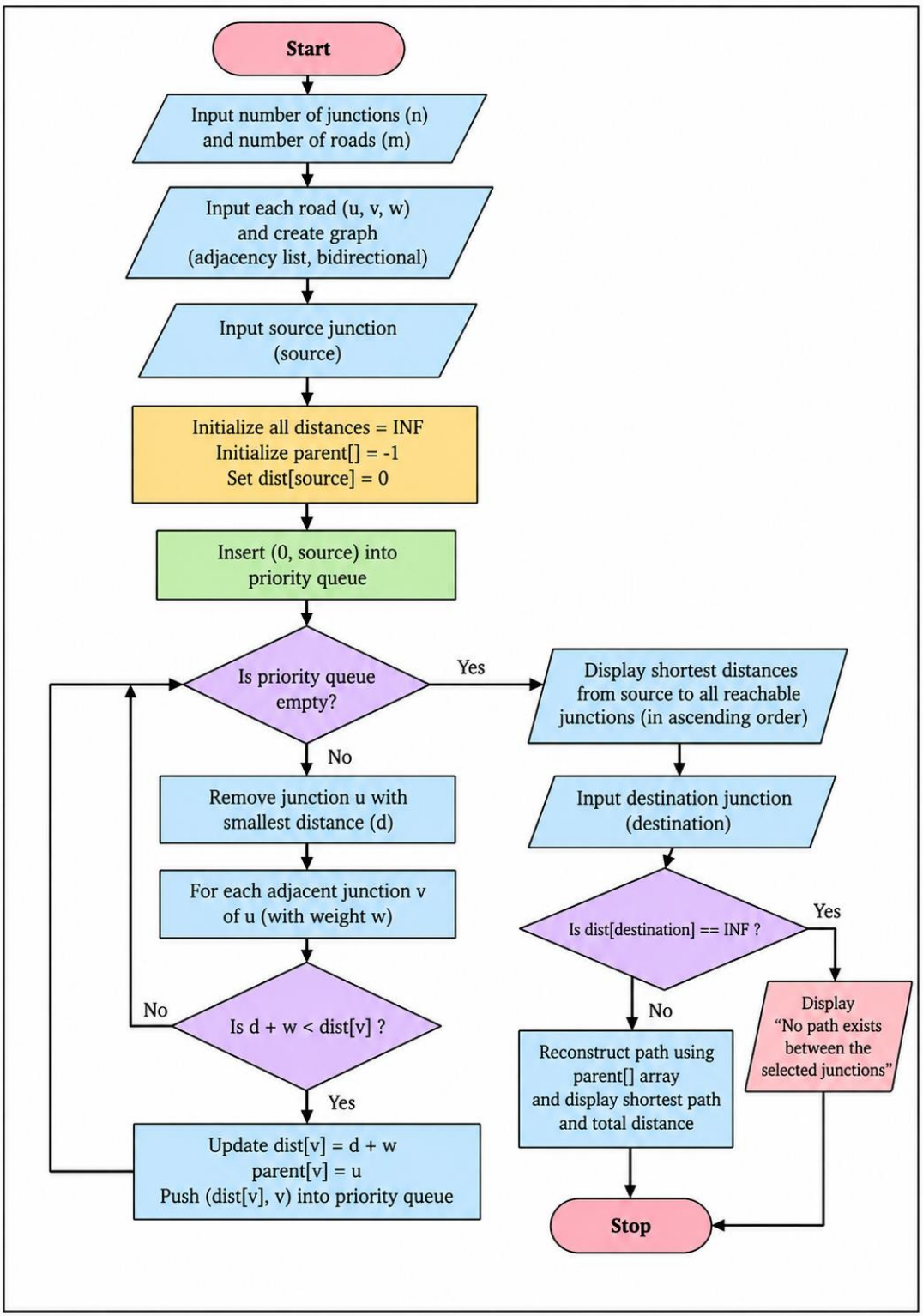
Objectives

1. To represent the smart-city road network as a weighted graph using an adjacency list.
2. To implement Dijkstra's algorithm for non-negative road distances.
3. To calculate the shortest distance from a source junction to every reachable junction.
4. To display reachable junctions in ascending order of their shortest distances.
5. To reconstruct and display the shortest path to a selected destination junction.
6. To validate user inputs and report when a destination cannot be reached.

Algorithm

1. Start the program.
2. Read the number of junctions (n) and roads (m). Validate the values.
3. Create an adjacency list with n + 1 entries, using junction numbers from 1 to n.
4. Read each road as (u, v, w). Add the road in both directions because roads are bidirectional. Reject invalid junction numbers or negative distances.
5. Read the source junction and initialize every distance to infinity and every parent to -1. Set the source distance to 0.
6. Insert (0, source) into a min-priority queue.
7. While the priority queue is not empty, remove the junction u with the smallest tentative distance.
8. Skip the entry if it contains an outdated distance. Otherwise, examine every adjacent junction v.
9. Calculate newDist = dist[u] + weight. If newDist is smaller than dist[v], update dist[v] and parent[v], then insert the new pair into the priority queue.
10. Collect all reachable junctions and sort them by distance. Display each junction and its shortest distance.
11. Read the destination junction. If its distance is infinity, display that no path exists.
12. Otherwise, follow the parent array from the destination back to the source, reverse the sequence, and display the shortest path and total distance.
13. Stop the program.

- **Flowchart**


Start
Input number of junctions (n) and number of roads (m)
Input each road (u, v, w) and create graph (adjacency list, bidirectional)
Input source junction (source)
Initialize all distances = INF
Initialize parent[] = -1
Set dist[source] = 0
Insert (0, source) into priority queue
Is priority queue empty?
Yes
No
Remove junction u with smallest distance (d)
For each adjacent junction v of u (with weight w)
Is d + w < dist[v] ?
No
Yes
Update dist[v] = d + w
parent[v] = u
Push (dist[v], v) into priority queue
Display shortest distances from source to all reachable junctions (in ascending order)
Input destination junction (destination)
Is dist[destination] == INF ?
Yes
No
Display "No path exists between the selected junctions"
Reconstruct path using parent[] array and display shortest path and total distance
Stop

- ## C++ Code

```
#include <iostream>
#include <vector>
#include <queue>
#include <algorithm>
#include <functional>

using namespace std;

const long long INF = 4000000000000000000LL;

void dijkstra(
    const vector<vector<pair<int, long long> > >& graph,
    int source,
    vector<long long>& dist,
    vector<int>& parent
) {
    int n = (int)graph.size() - 1;

    dist.assign(n + 1, INF);
    parent.assign(n + 1, -1);

    priority_queue<
        pair<long long, int>,
        vector<pair<long long, int> >,
        greater<pair<long long, int> >
    > pq;

    dist[source] = 0;
    pq.push(make_pair(0LL, source));

    while (!pq.empty()) {
        long long currentDist = pq.top().first;
        int u = pq.top().second;
        pq.pop();

        if (currentDist != dist[u])
            continue;

        vector<pair<int, long long> >::const_iterator it;

        for (it = graph[u].begin(); it != graph[u].end(); ++it) {
            int v = it->first;
            long long weight = it->second;

            if (currentDist > INF - weight)
                continue;

            long long newDist = currentDist + weight;

            if (newDist < dist[v]) {
                dist[v] = newDist;
                parent[v] = u;
                pq.push(make_pair(newDist, v));
            }
        }
    }
}

int main() {
    int n, m;

    cout << "Enter number of junctions: ";
    if (!(cin >> n) || n <= 0) {
        cout << "Invalid number of junctions.\n";
        return 1;
    }

    cout << "Enter number of roads: ";
    if (!(cin >> m) || m < 0) {
        cout << "Invalid number of roads.\n";
        return 1;
    }

    vector<vector<pair<int, long long> > > graph(n + 1);

    cout << "\nEnter roads as: junction1 junction2 distance\n";
    cout << "Roads are bidirectional.\n";

    for (int i = 0; i < m; i++) {
        int u, v;
        long long w;
```

```
        cout << "Road " << i + 1 << ": ";

        if (!(cin >> u >> v >> w) ||
            u < 1 || u > n ||
            v < 1 || v > n ||
            w < 0 || w >= INF) {
            cout << "Invalid road details.\n";
            return 1;
        }

        graph[u].push_back(make_pair(v, w));
        graph[v].push_back(make_pair(u, w));
    }

    int source, destination;

    cout << "\nEnter source junction: ";
    if (!(cin >> source) || source < 1 || source > n) {
        cout << "Invalid source junction.\n";
        return 1;
    }

    vector<long long> dist;
    vector<int> parent;

    dijkstra(graph, source, dist, parent);

    vector<pair<long long, int> > reachable;

    for (int i = 1; i <= n; i++) {
        if (dist[i] != INF) {
            reachable.push_back(make_pair(dist[i], i));
        }
    }

    sort(reachable.begin(), reachable.end());

    cout << "\nShortest distances from junction "
         << source << ":\n";

    cout << "Junction\tDistance\n";

    for (int i = 0; i < (int)reachable.size(); i++) {
        cout << reachable[i].second << "\t\t"
             << reachable[i].first << "\n";
    }

    cout << "\nEnter destination junction: ";
    if (!(cin >> destination) ||
        destination < 1 || destination > n) {
        cout << "Invalid destination junction.\n";
        return 1;
    }

    if (dist[destination] == INF) {
        cout << "No path exists between the selected junctions.\n";
        return 0;
    }

    vector<int> path;

    int v = destination;

    while (v != -1) {
        path.push_back(v);
        v = parent[v];
    }

    reverse(path.begin(), path.end());

    cout << "\nShortest path: ";

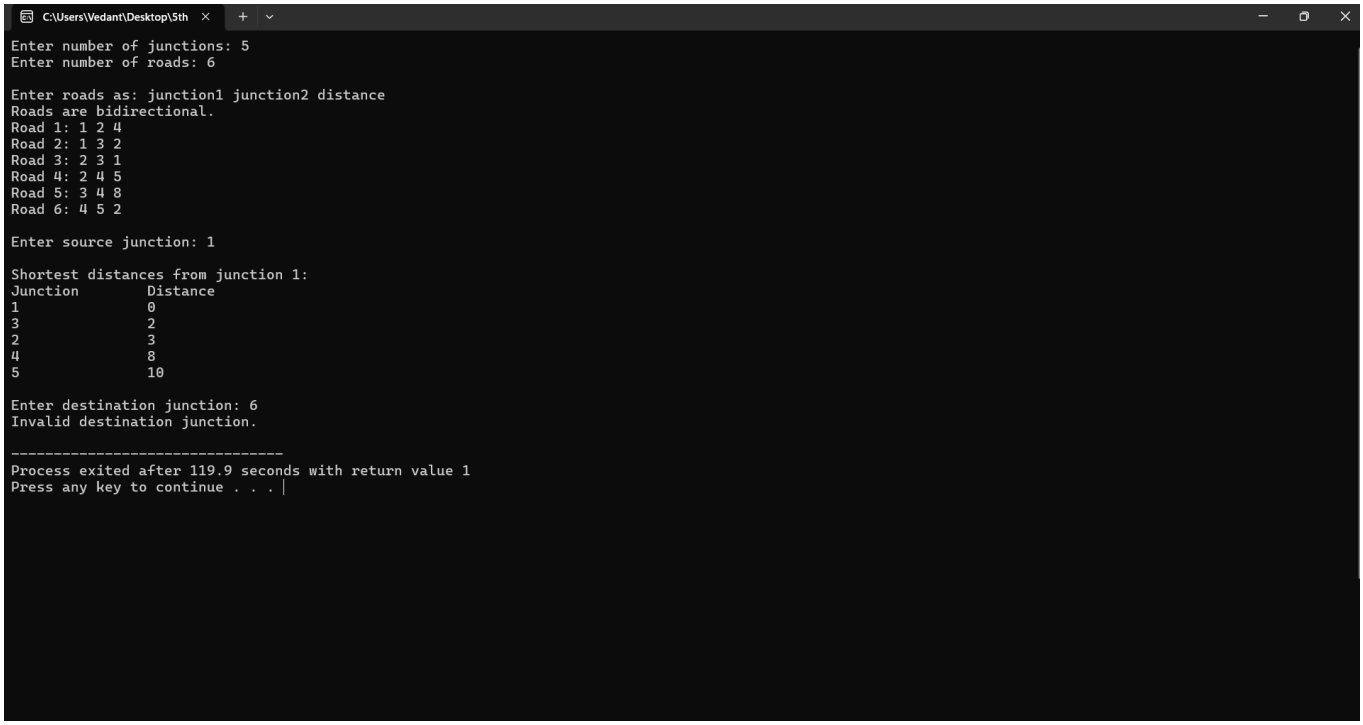
    for (int i = 0; i < (int)path.size(); i++) {
        cout << path[i];

        if (i < (int)path.size() - 1)
            cout << " -> ";
    }

    cout << "\nTotal distance: " << dist[destination] << "\n";

    return 0;
}
```

- **Code Output**


C:\Users\Vedant\Desktop\5th
Enter number of junctions: 5
Enter number of roads: 6
Enter roads as: junction1 junction2 distance
Roads are bidirectional.
Road 1: 1 2 4
Road 2: 1 3 2
Road 3: 2 3 1
Road 4: 2 4 5
Road 5: 3 4 8
Road 6: 4 5 2
Enter source junction: 1
Shortest distances from junction 1:
Junction Distance
1 0
3 2
2 3
4 8
5 10
Enter destination junction: 6
Invalid destination junction.
Process exited after 119.9 seconds with return value 1
Press any key to continue . . .